

1.) $N * (\log_2 N)^2 + N^2 * (\log_2 N)$

↓
leading term.

$\Rightarrow \boxed{c)} \Theta(N^2 \cdot \log_2 N)$

$\Rightarrow \boxed{d)} O(N^3)$

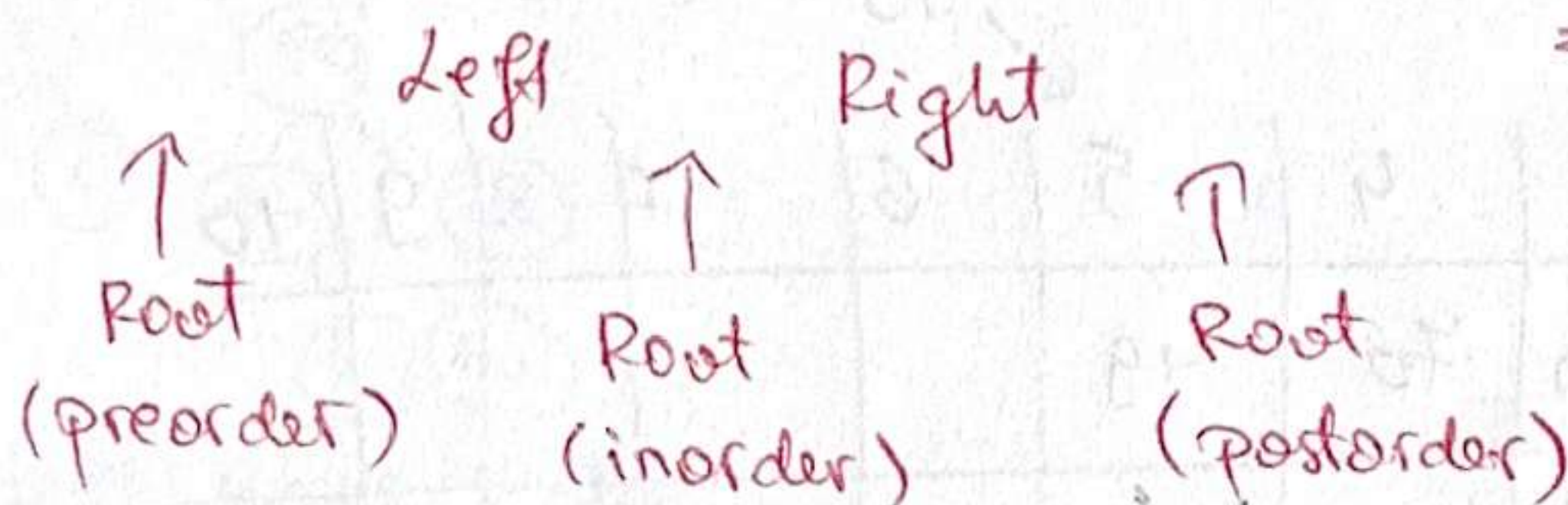
$\Rightarrow \boxed{a)} \Omega(N \cdot \log_2 N)$

2) inorder: $\overbrace{B} \text{ } \overbrace{A} \text{ } \overbrace{D \ E \ C \ F}$
preorder: $\overbrace{A} \text{ } \overbrace{B \ C \ D \ E \ F}$

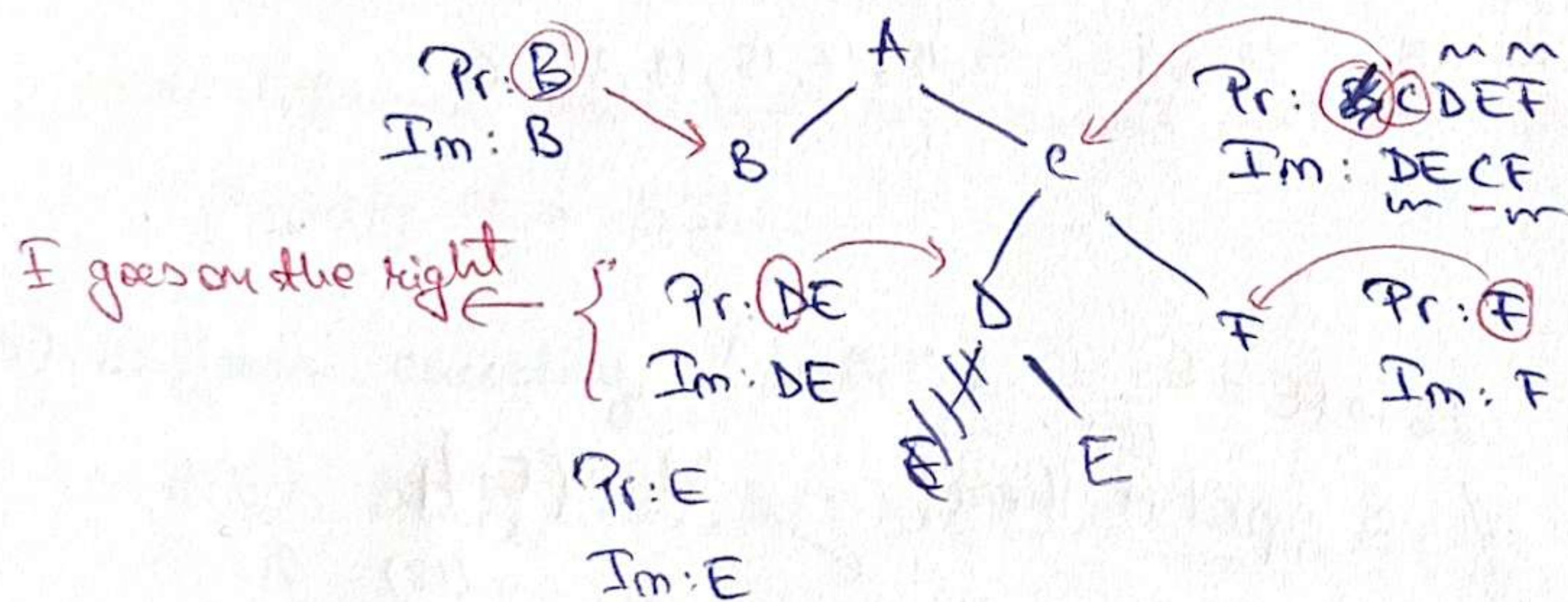
leaves?

(Lecture 13)

$\Rightarrow$ Preorder gives the ROOT.



$\Rightarrow A = \text{root}$.



$\Rightarrow$ leaves: B, E, F $\Rightarrow \boxed{b)}, \boxed{e)}, \boxed{f)}$

3) HT, 100 positions, collision res., open addressing; max. nr. of elements that can be inserted?

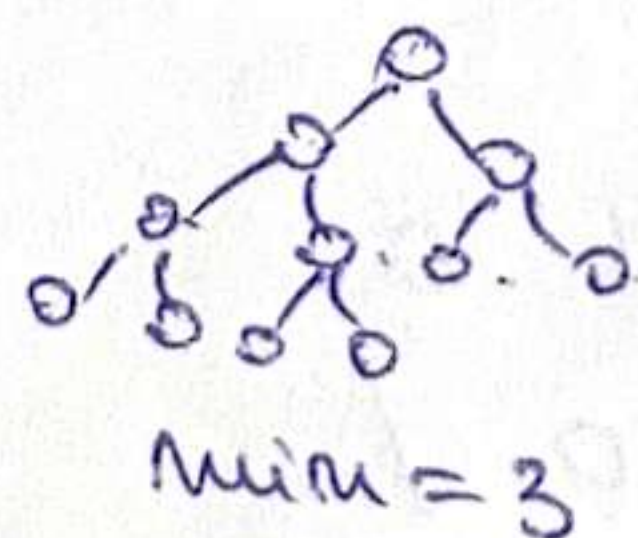
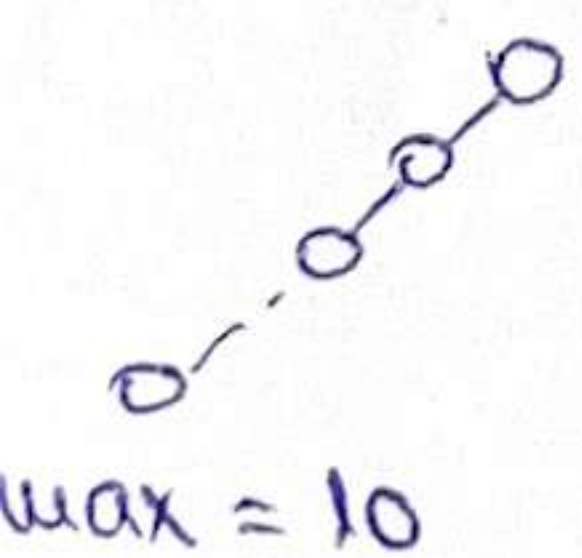
Open addressing $\rightarrow$ each elem. stored inside the table itself.
 $\rightarrow$ insertion continues until an empty slot is found
 $\rightarrow$ table holds at most 1 elem/pos.

$\Rightarrow \boxed{a) 100}$

4) max-height - min-height of a BT with 11 nodes?

BT = has max 2 children

- can have "unfilled" levels ($\neq$ heap must have all levels filled)



$\Rightarrow 10 - 3 = 7 \Rightarrow \boxed{d)}$

compute $\Rightarrow$ push result back to the stack

B. 2.1) HT, size $m=11$, collision res, coalesced chaining, div. method

→ insert: 49, 13, 93, 66, 23, 57, 70, 110.

→ how many elems. are not on position where they hash to.

→ insertion order to minimise collisions → explain idea.

$$h(49) = 49 \% 11 = 5 \checkmark$$

$$h(13) = 13 \% 11 = 2 \checkmark$$

$$h(93) = 5$$

first empty: 3/6

$$h(66) = 0 \checkmark$$

$$h(23) = 1 \checkmark$$

$$h(57) = 2$$

$$h(70) = 4 \checkmark$$

$$h(110) = 0$$

pos	0	1	2	3	4	5	6	7	8	9	10
T	66	23	13	93	70	49					
next	-1	-1	X	X	-1	X	-1	-1	-1	-1	-1

Another example: $m=13$; insert: 5, 18, 16, 15, 13, 31, 26

$$h(5) = 5 \checkmark$$

$$h(18) = 5 \sim$$

$$h(16) = 3 \checkmark$$

$$h(15) = 2 \checkmark$$

$$h(13) = 0 \sim$$

$$h(31) = 5 \checkmark$$

$$h(26) = 0$$

$$26 \rightarrow fE = 6$$

put it there!

$$31 \rightarrow fE = 4$$

$$18 \rightarrow \text{first Empty: } 0 \rightarrow \text{put it there!}$$

pos	0	1	2	3	4	5	6	7	8	9	10	11	12
T	18	13	15	16	31	5	26						
next	X	X	X	X	X	X	X	-1	-1	-1	-1	-1	-1

$$110 \rightarrow fE = 7$$

$$1$$

$$66 \rightarrow fE = 1$$

$$23 \rightarrow fE = 3$$

$$57 \rightarrow fE = 4$$

Again:

pos	0	1	2	3	4	5	6	7	8	9	10
T	93	66	13	23	57	49	70	110			
next	X	X	X	X	X	X	X	X	-1	-1	-1

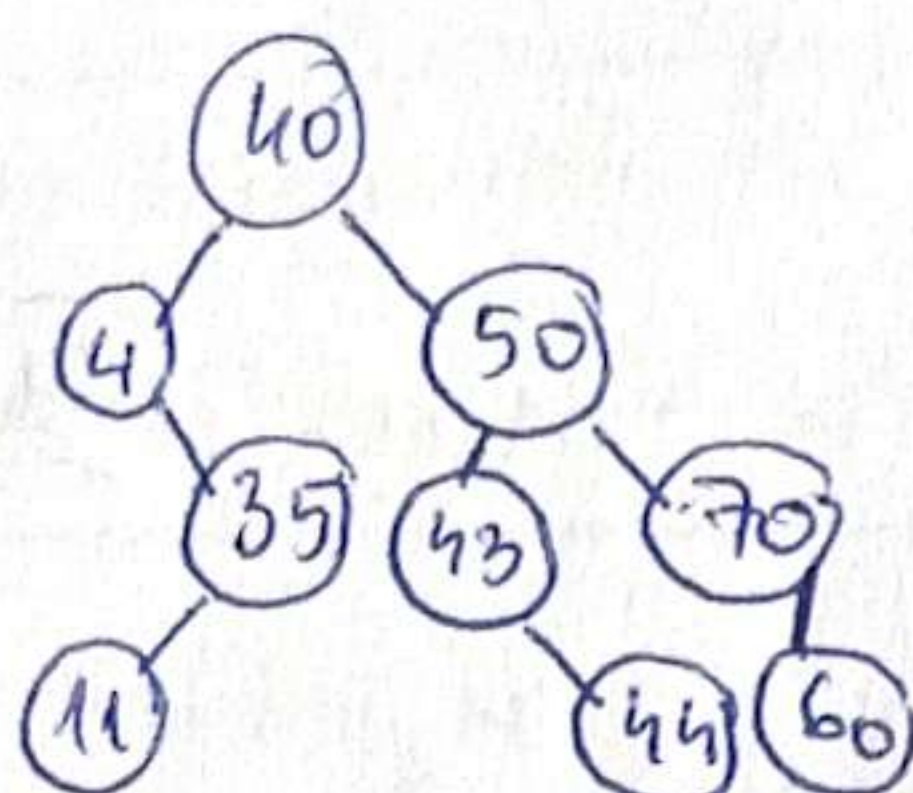
first empty: 0, 3, 4, 6, 7, 8

⇒ 6 elems are not on pos. they hash to.

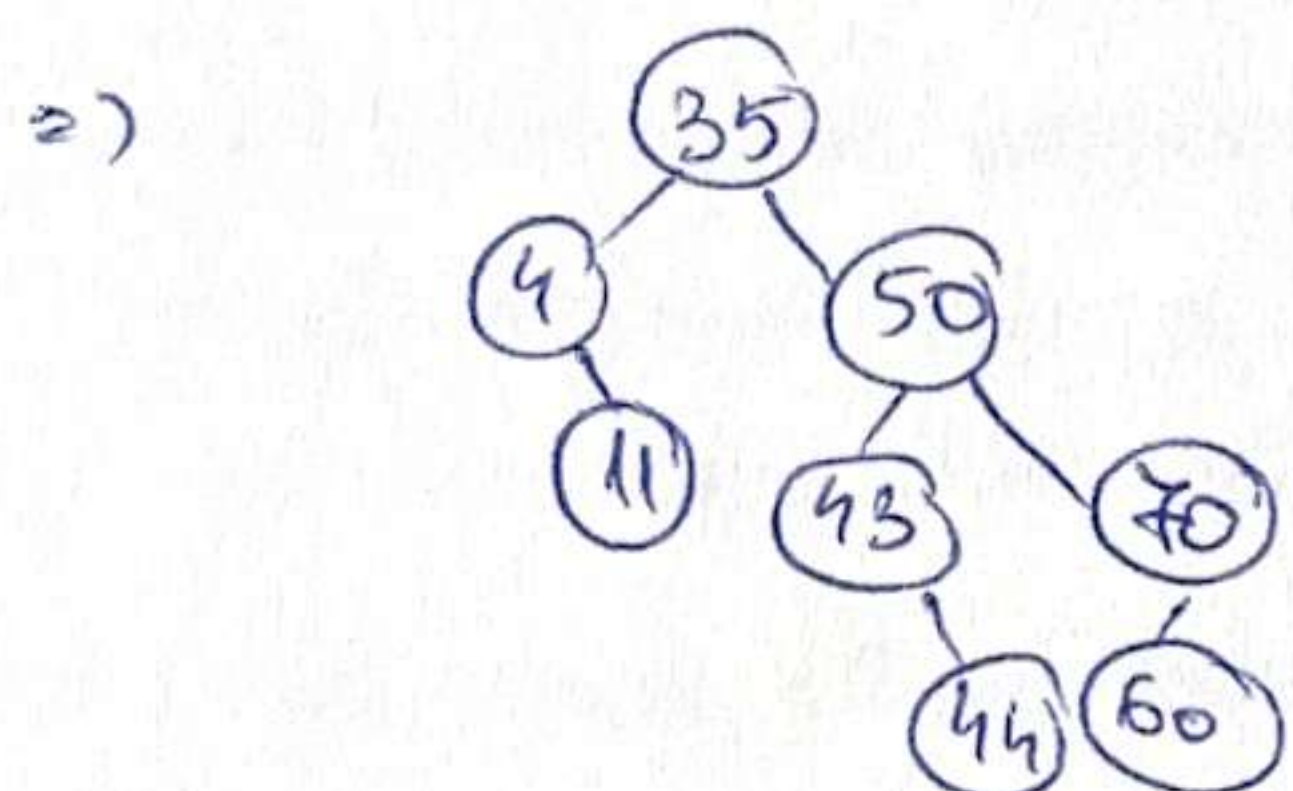
Insertion order: 49, 13, 66, 23, 70, 93, 57, 110. ✓

BST $\rightarrow$ insert: 40, 4, 35, 11, 50, 70, 43, 44, 60.

BST = BT + ($\text{left} \leq \text{current} \leq \text{right}$) + for sorted containers.



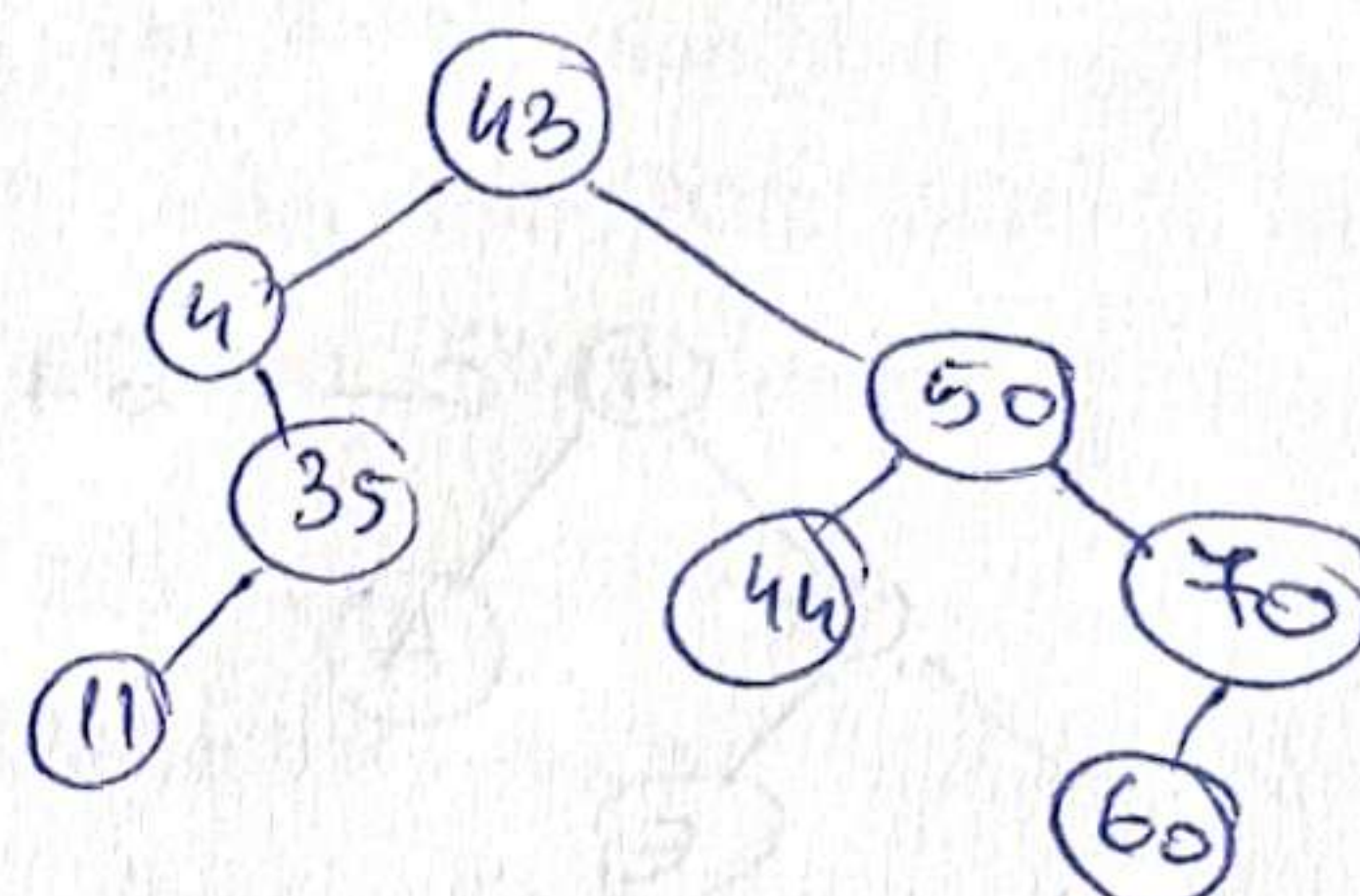
replace with predecessor =
35 = max value in the left subtree



inorder: left Root Right

4 11 35 43 44 50 60 70

replace with successor =
43 = min value in the right subtree



inorder: 4 11 35 43 44 50 60 70

=> no difference because both replacement methods preserve the logical sorted property of a BST.

2.3) Huffman encoding (Lect. 13)

A(5), B(2), C(8), D(7), E(3), F(7), G(1).

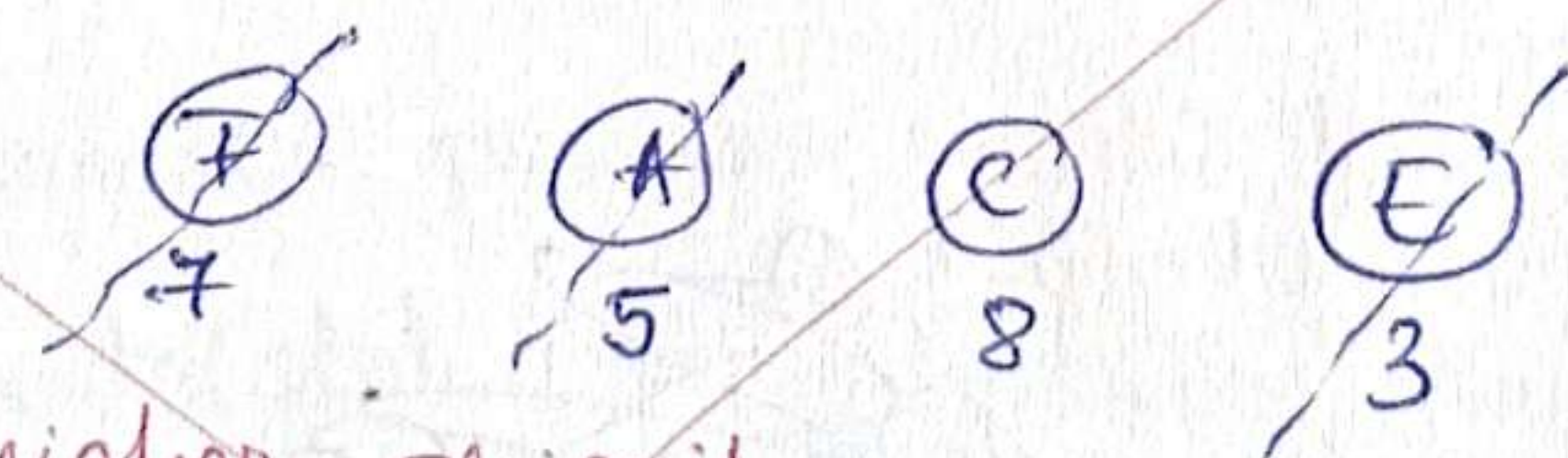
↑ ↑
letter freq.

→ process of building the tree

→ code for every letter.

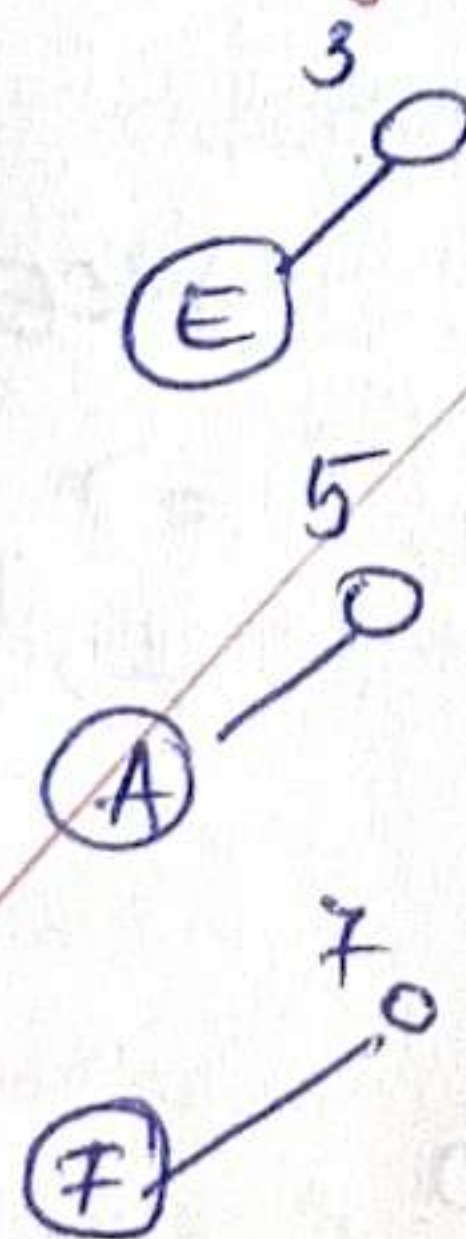
→ encode: FACE ; length = ?

=> priority queue.



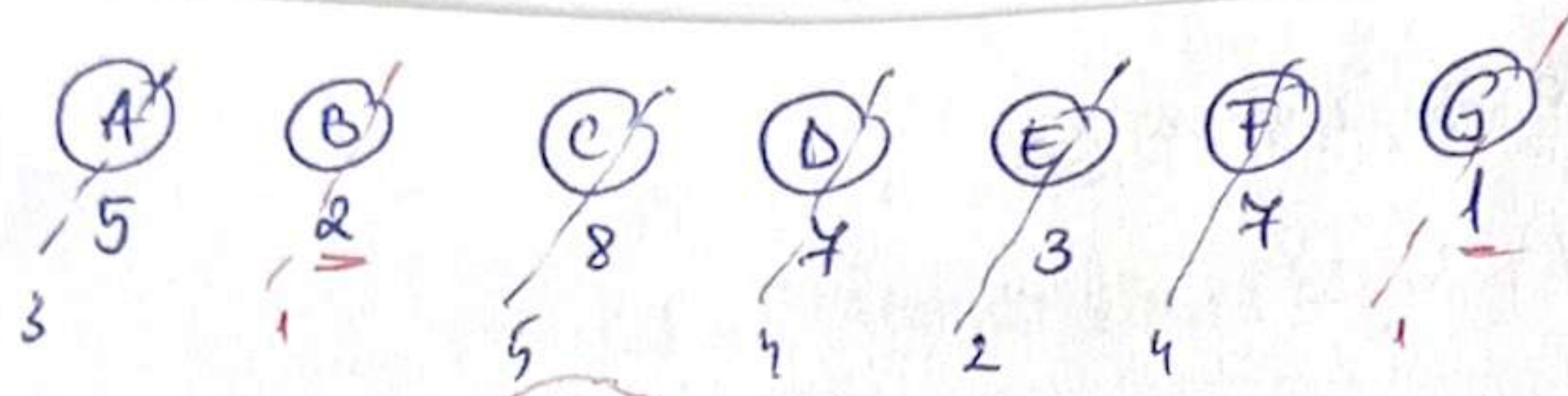
from the initial list.

smaller freq. => higher priority

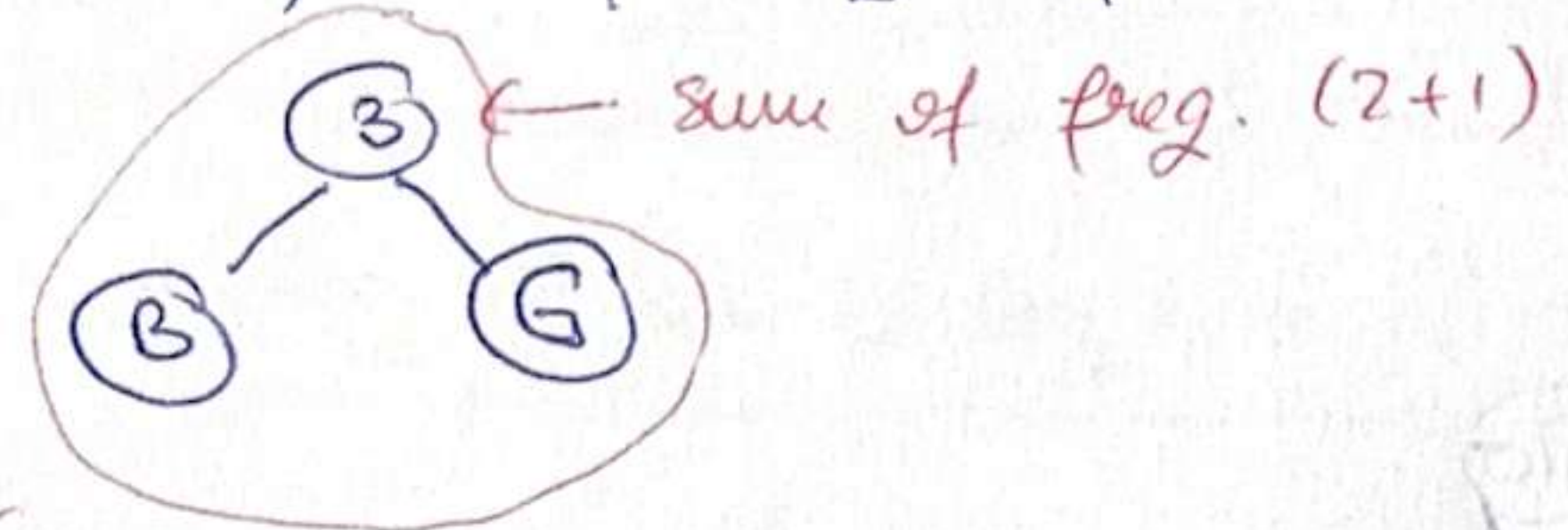


3 = sum of the priorities of the children

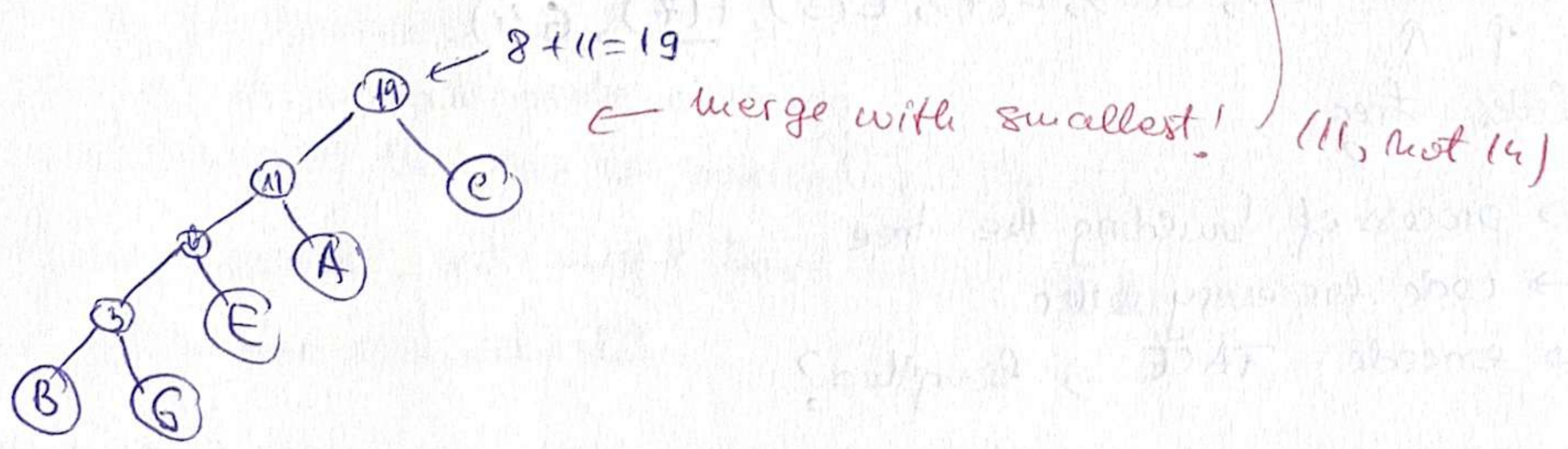
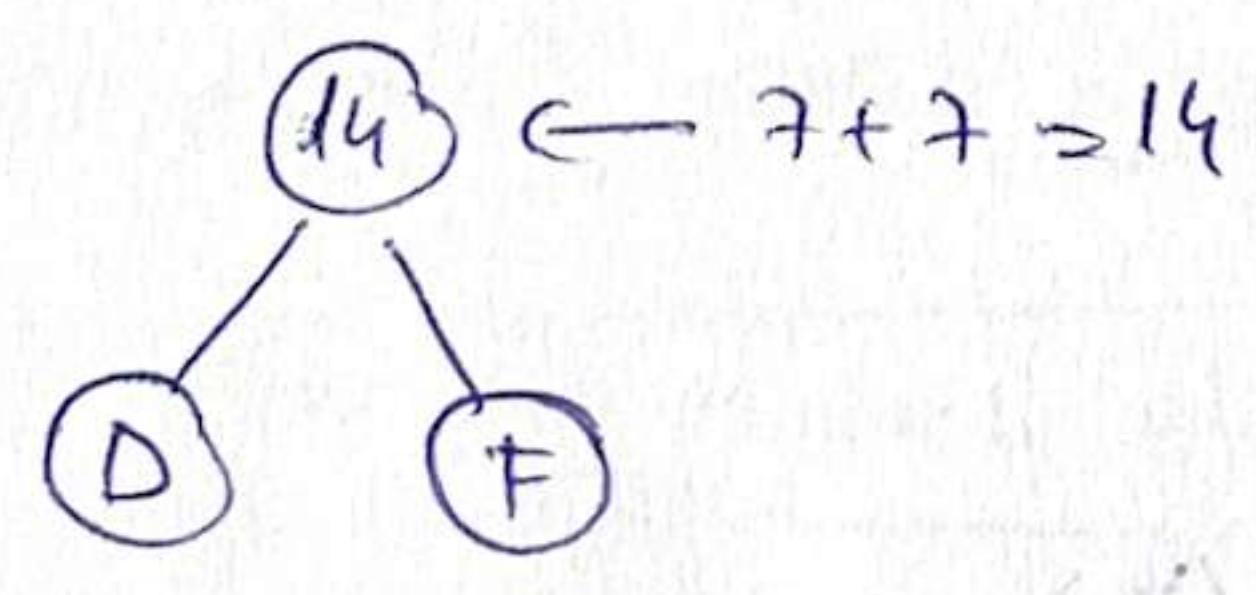
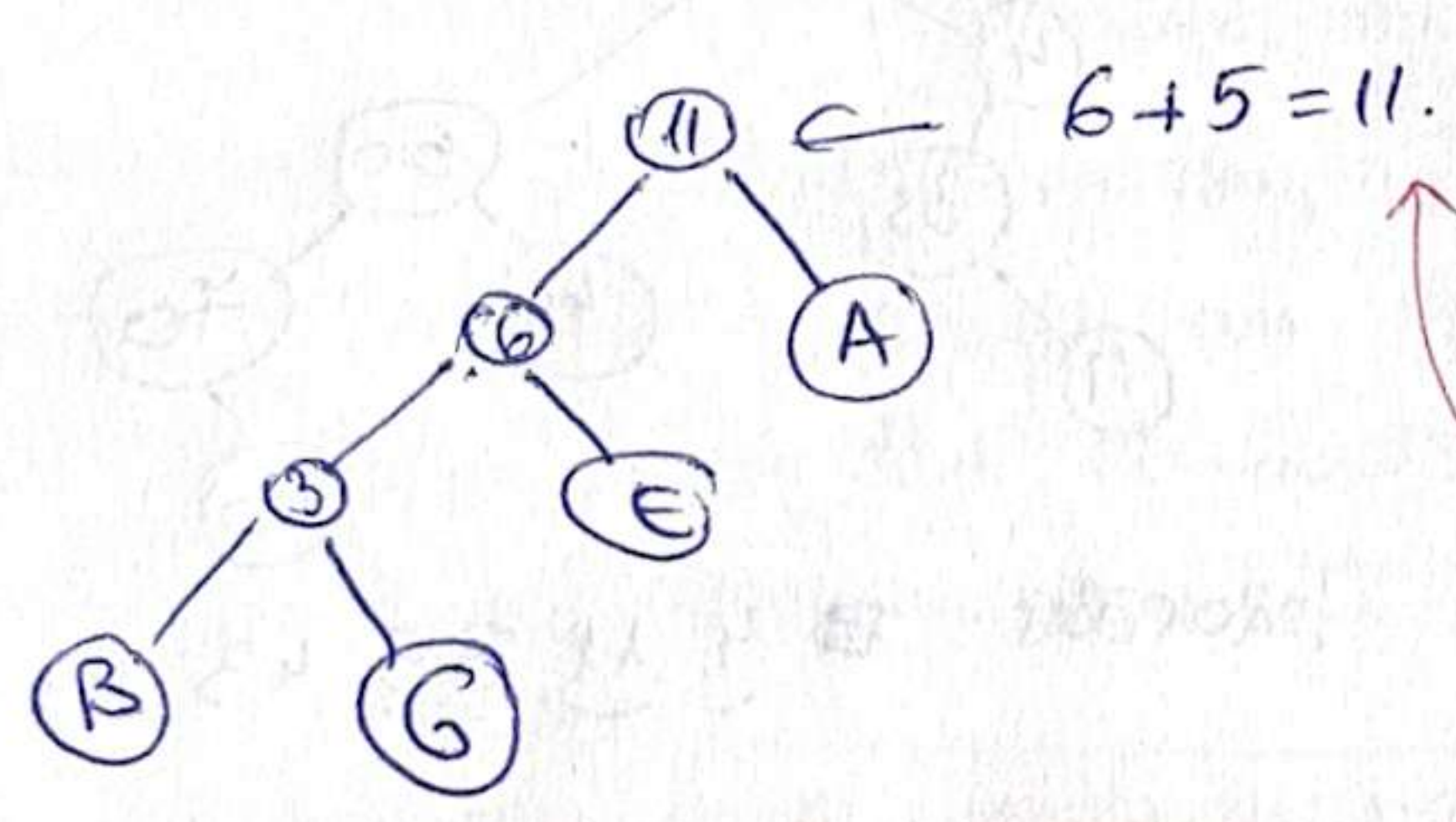
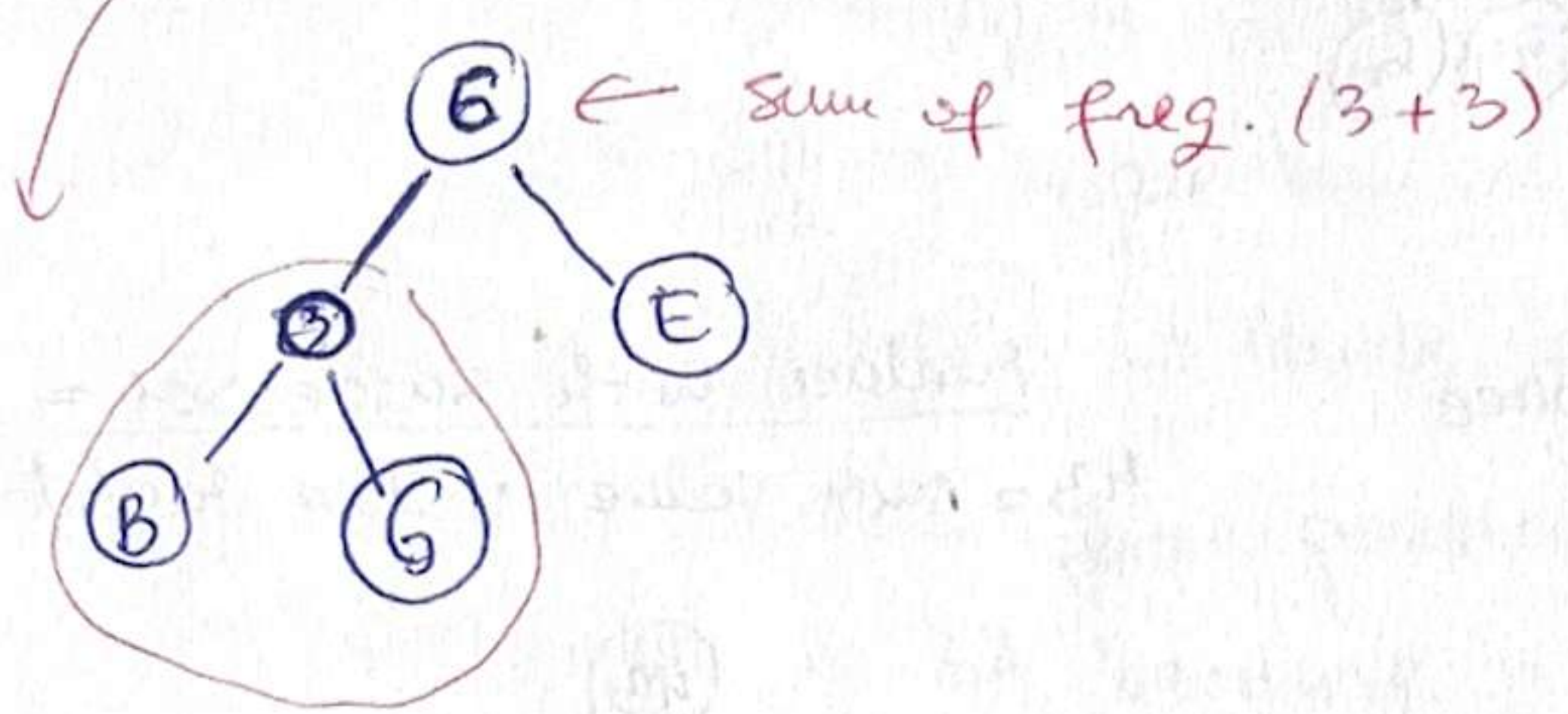
smaller freq. first!



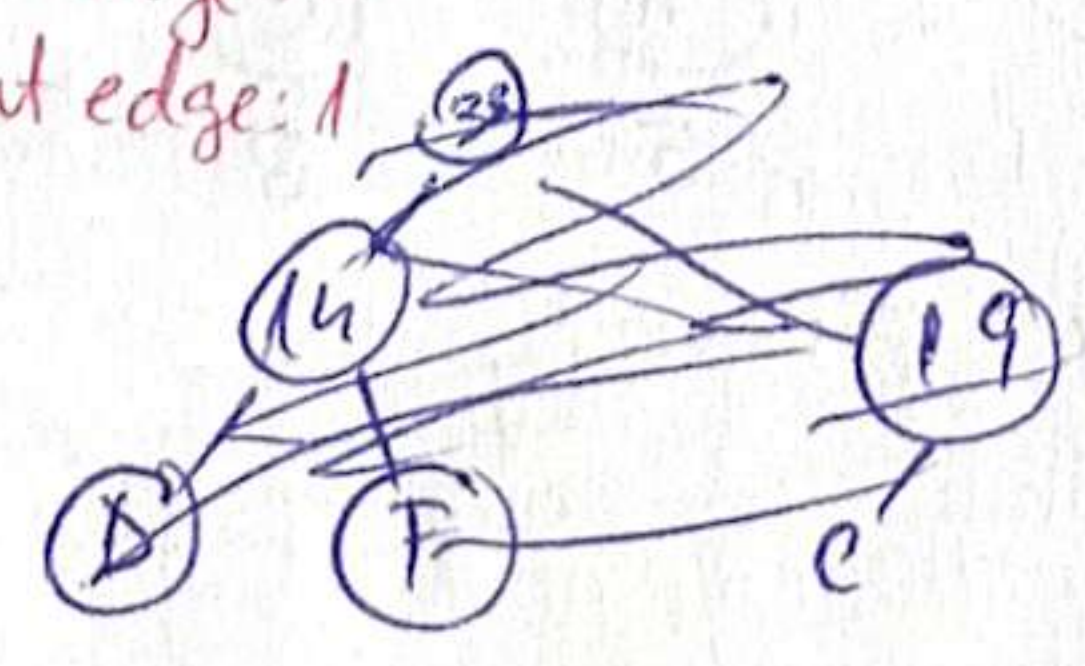
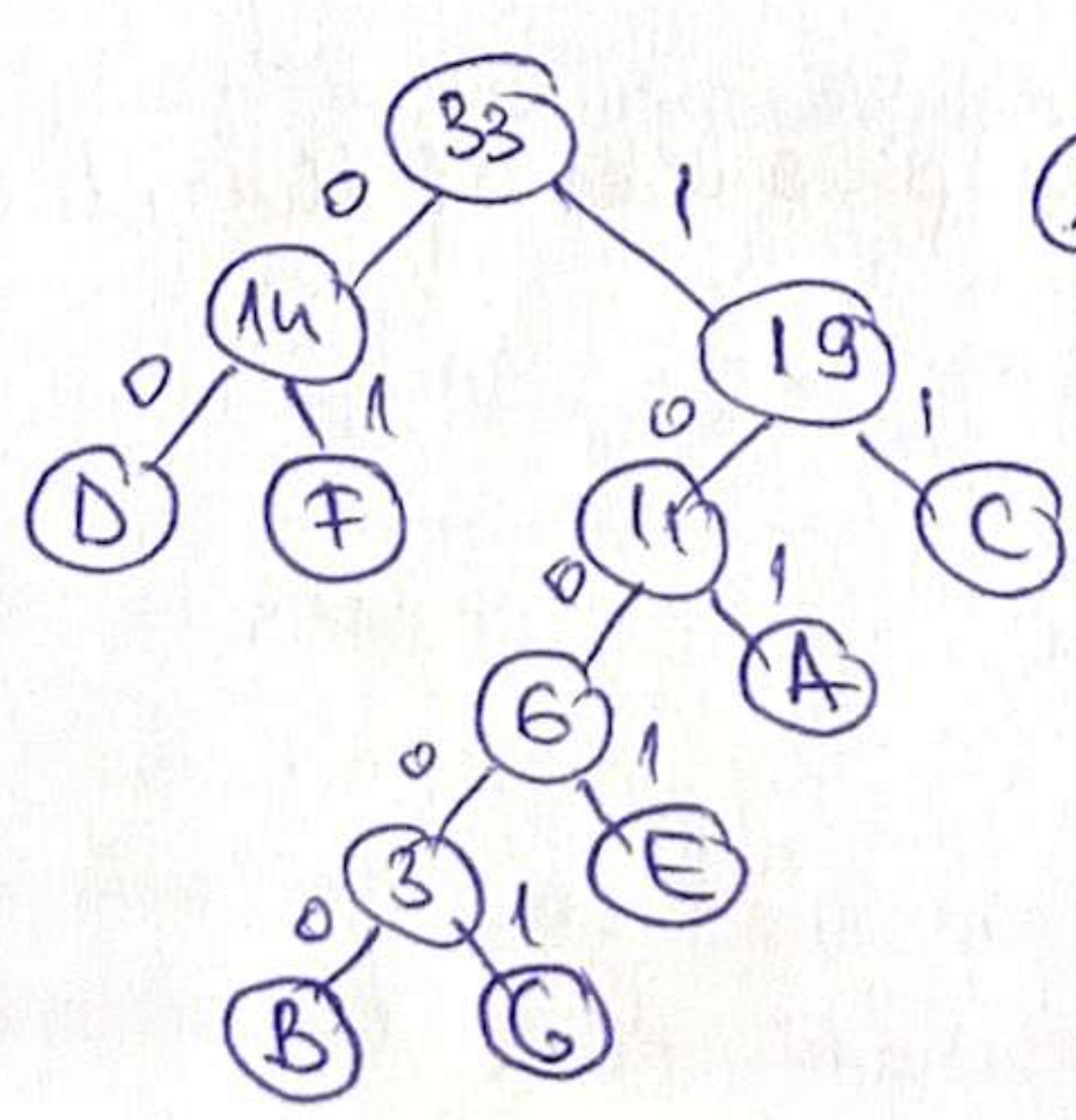
on the left!



(u, 3), 4) same as row 12



Final merge:
left edge: 0
right edge: 1



- D : 00
- F : 01
- C : 11
- A : 101
- E : 1001
- G : 10001
- B : 10000

FACE: 01101111001
 2) 01101111001
 2) length = 11